

AGENTS.md - Challenges Module

INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitution’s AGENTS.md and the Constitution.md it references apply unconditionally. Locate the constitution from any nested depth via its find_constitution.sh helper – do NOT hardcode a path (this module stays fully decoupled and project-agnostic per §11.4.28).

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

Module Overview

digital.vasic.challenges is a generic, reusable Go module for defining, registering, executing, and reporting on challenges (structured test scenarios). It provides a comprehensive framework for validation testing with built-in assertion evaluation, multiple reporting formats, live monitoring, and plugin extensibility.

Module path: digital.vasic.challenges **Go version:** 1.24+ **Dependencies:** digital.vasic.containers (infrastructure bridge), standard Go libraries, testify (tests only)

Package Responsibilities

Package	Path	Responsibility
challenge	pkg/ challenge/	Core types and interfaces: Challenge interface defining lifecycle (Configure, Validate, Execute, Cleanup). BaseChallenge template implementation. Config and Result structures. Challenge status enumeration.
registry	pkg/ registry/	Challenge registration and dependency management: Registry with topological sorting (Kahn’s algorithm). Dependency validation (cycle detection). Challenge lookup by ID/tags. Ordered execution sequencing.
runner	pkg/runner/	Execution engine: Sequential, parallel, and pipeline execution modes. Concurrent execution with semaphore control. Timeout handling and graceful cancellation. Result aggregation and reporting.
assertion	pkg/ assertion/	Assertion evaluation engine: 16 built-in evaluators (not_empty, contains, min_length, quality_score, etc.). Custom evaluator registration. Expression parser for complex assertions. Flexible comparison operators.
report	pkg/report/	Report generation: Multiple formats (Markdown, JSON, HTML). Summary statistics (pass/fail/

Package	Path	Responsibility
		skip counts). Detailed execution timelines. Export to file or string.
logging	pkg/logging/	Structured logging: JSON and console formatters. Multi-logger composition. Redacting logger for sensitive data. API request/response tracking.
env	pkg/env/	Environment variable management: Load from .env files. Variable interpolation and defaults. Redaction patterns for secrets (API keys, tokens). Validation and type conversion.
bank	pkg/bank/	Challenge bank (definition loading): Load challenge definitions from JSON/YAML. Template variable substitution. Bulk challenge instantiation. Definition validation.
monitor	pkg/monitor/	Live monitoring: WebSocket-based real-time dashboard. Event collection (challenge start/complete/fail). Progress tracking and ETA calculation. Metrics export.
metrics	pkg/metrics/	Prometheus-compatible metrics: Challenge execution counters. Duration histograms. Success/failure rates. Custom metric registration.
plugin	pkg/plugin/	Plugin system: Plugin interface for custom challenge types. Dynamic plugin loading. Lifecycle management (Init, Shutdown). Plugin registry and versioning.
infra	pkg/infra/	Infrastructure bridge: Adapter to <code>digital.vasic.containers</code> module. Service startup/shutdown coordination. Health check integration. Resource cleanup.

Dependency Graph

```

runner  ---> challenge, registry, assertion, report, logging
registry ---> challenge
assertion ---> logging
report ---> challenge, logging
monitor ---> challenge, metrics
plugin ---> challenge, registry
infra ---> challenge, containers module
bank ---> challenge, env

```

challenge is the foundational package. runner integrates most packages for orchestration.

Key Files

File	Purpose
pkg/challenge/ challenge.go	Challenge interface, BaseChallenge, Config, Result types
pkg/challenge/shell.go	ShellChallenge for executing bash scripts

File	Purpose
pkg/registry/registry.go	Registry implementation with dependency ordering
pkg/runner/runner.go	Runner implementation with execution modes
pkg/assertion/engine.go	Assertion engine with built-in evaluators
pkg/assertion/ evaluators.go	All 16 built-in evaluator implementations
pkg/report/markdown.go	Markdown report generator
pkg/report/json.go	JSON report generator
pkg/report/html.go	HTML report generator
pkg/logging/logger.go	Logger interface and implementations
pkg/env/loader.go	Environment variable loader with redaction
pkg/bank/bank.go	Challenge bank with JSON/YAML loading
pkg/monitor/monitor.go	Live monitoring with WebSocket server
pkg/metrics/metrics.go	Prometheus metrics collector
pkg/plugin/plugin.go	Plugin interface and registry
pkg/infra/adapter.go	Containers module adapter
go.mod	Module definition and dependencies
CLAUDE.md	AI coding assistant instructions
README.md	User-facing documentation with quick start

Agent Coordination Guide

Division of Work

When multiple agents work on this module simultaneously, divide work by package boundary:

1. **Challenge Agent** â€œ Owns pkg/challenge/. Core types affect all other packages. Must coordinate before modifying Challenge interface or Result structure.
2. **Registry Agent** â€œ Owns pkg/registry/. Dependency ordering logic. Changes rarely affect other packages except runner.
3. **Runner Agent** â€œ Owns pkg/runner/. Integration layer. Requires testing against all execution modes.
4. **Assertion Agent** â€œ Owns pkg/assertion/. New evaluators can be added independently. Evaluator registry changes require runner updates.
5. **Report Agent** â€œ Owns pkg/report/. New report formats can be added independently. Reporter interface changes affect runner.
6. **Monitoring Agent** â€œ Owns pkg/monitor/. Real-time monitoring. Can work independently but coordinates with runner for event hooks.
7. **Plugin Agent** â€œ Owns pkg/plugin/. Plugin system. Must coordinate with registry for plugin registration.

Coordination Rules

- **Challenge interface changes** require all agents to update. The Challenge interface is the shared contract.
- **Assertion evaluators** and **report formats** are independent and can be modified in parallel.
- **Runner package** integrates all packages. Any interface change in sub-packages requires corresponding runner updates.
- **Monitor and metrics** packages are loosely coupled. Coordinate on event schema.

- **Test isolation:** Each package has its own `_test.go` files. Integration tests in `runner` package.
- **No circular dependencies:** The dependency graph is strictly acyclic. Never import `runner` from sub-packages.

Safe Parallel Changes

These changes can be made simultaneously without coordination: - Adding a new assertion evaluator to `pkg/assertion/` - Adding a new report format to `pkg/report/` - Adding new monitoring events to `pkg/monitor/` - Adding new plugins to `pkg/plugin/` - Adding new tests to any package - Updating documentation

Changes Requiring Coordination

- Modifying the `Challenge` interface methods
- Changing `Result` structure fields
- Modifying assertion evaluator registry interface
- Adding new execution modes to `runner`
- Changing event schema in `monitor`
- Modifying plugin interface

Build and Test Commands

```
# Build all packages
go build ./...

# Run all tests with race detection
go test ./... -count=1 -race

# Run unit tests only (short mode)
go test ./... -short

# Run integration tests (requires Containers module)
go test -tags=integration ./...

# Run benchmarks
go test -bench=. ./tests/benchmark/

# Run a specific test
go test -v -run TestRunner_RunAll ./pkg/runner/

# Format code
gofmt -w .

# Vet code
go vet ./...
```

Commit Conventions

Follow Conventional Commits with package scope:

```

feat(assertion): add regex_match evaluator
feat(report): add PDF report generator
feat(plugin): implement plugin versioning
fix(runner): prevent race condition in parallel execution
fix(registry): detect circular dependencies correctly
test(assertion): add evaluator edge case tests
docs(challenges): update plugin development guide
refactor(monitor): extract WebSocket server to separate file

```

Thread Safety Notes

- **Runner** executes challenges concurrently with semaphore control. Uses `sync.WaitGroup` for coordination.
- **Registry** is thread-safe for reads after initialization. Writes during registration use mutex.
- **Assertion engine** evaluators must be safe for concurrent invocation.
- **Monitor** uses channels for event collection and mutex for state access.
- **Metrics collector** uses atomic operations for counters.
- **Plugin registry** locks during plugin loading/unloading.

Built-in Assertion Evaluators

Evaluator	Description	Example
not_empty	Value must not be empty string or nil	"result": { "not_empty": true }
not_mock	Response must not be a mock/placeholder	"response": { "not_mock": true }
contains	String contains substring	"output": { "contains": "success" }
contains_any	String contains any of the substrings	"output": { "contains_any": ["ok", "success"] }
min_length	String has minimum length	"response": { "min_length": 10 }
quality_score	LLM response quality score (0-1)	"quality": { "quality_score": 0.8 }
reasoning_present	Response contains reasoning/explanation	"answer": { "reasoning_present": true }
code_valid	Code block is syntactically valid	"code": { "code_valid": "go" }
min_count	Array/collection minimum count	"items": { "min_count": 5 }
exact_count	Array/collection exact count	"items": { "exact_count": 10 }
max_latency	Operation completed within time limit (ms)	"latency": { "max_latency": 1000 }

Evaluator	Description	Example
all_valid	All items in array pass validation	"results": { "all_valid": true }
no_duplicates	Array has no duplicate values	"ids": { "no_duplicates": true }
all_pass	All nested assertions pass	"tests": { "all_pass": true }
no_mock_responses	No mock/placeholder responses in collection	"responses": { "no_mock_responses": true }
min_score	Numeric score meets minimum threshold	"score": { "min_score": 85.0 }

Configuration Example

```
package main

import (
    "context"
    "digital.vasic.challenges/pkg/challenge"
    "digital.vasic.challenges/pkg/registry"
    "digital.vasic.challenges/pkg/runner"
    "digital.vasic.challenges/pkg/report"
)

func main() {
    // Create registry
    reg := registry.New()

    // Register challenges
    reg.Register(&MyChallenge{
        id: "test-api",
        dependencies: []string{},
    })

    // Create runner
    run := runner.New(reg,
        runner.WithParallelism(5),
        runner.WithTimeout(30*time.Second),
    )

    // Execute all challenges
    results, _ := run.RunAll(context.Background())

    // Generate report
    reporter := report.NewMarkdownReporter()
    report := reporter.Generate(results)
    fmt.Println(report)
}
```

Custom Challenge Example

```
type APITestChallenge struct {
    challenge.BaseChallenge
    endpoint string
}

func (c *APITestChallenge) Execute(ctx context.Context) error {
    resp, err := http.Get(c.endpoint)
    if err != nil {
        return err
    }
    defer resp.Body.Close()

    if resp.StatusCode != 200 {
        return fmt.Errorf("unexpected status: %d", resp.StatusCode)
    }

    // Evaluate assertions
    return c.AssertionEngine.Evaluate(ctx, map[string]interface{}{
        "status_code": resp.StatusCode,
        "response": map[string]interface{}{
            "not_empty": true,
            "contains": "success",
        },
    })
}
```

Best Practices

1. Use Dependency Ordering

```
// Good - declare dependencies
challenge := &MyChallenge{
    dependencies: []string{"setup-database", "start-api"},
}
```

// Registry will execute in correct order

2. Implement Cleanup

```
func (c *MyChallenge) Cleanup(ctx context.Context) error {
    // Always clean up resources
    c.client.Close()
    c.db.Close()
    return nil
}
```

3. Use Assertion Engine

```
// Good - use built-in evaluators
return c.AssertionEngine.Evaluate(ctx, map[string]interface{}{
```

```

        "result": {
            "not_empty": true,
            "min_length": 10,
        },
    })

// Bad - manual validation
if result == "" || len(result) < 10 {
    return errors.New("validation failed")
}

```

4. Set Timeouts

```

// Good - always set timeouts
runner := runner.New(reg,
    runner.WithTimeout(30*time.Second),
)

// Bad - no timeout (can hang indefinitely)
runner := runner.New(reg)

```

5. Use Structured Logging

```

// Good - structured logging
logger.InfoWithFields("Challenge executed", map[string]interface{}{
    "challenge_id": c.ID(),
    "duration": elapsed,
    "status": "passed",
})

// Bad - unstructured logging
log.Printf("Challenge %s took %v and passed", c.ID(), elapsed)

```

Last Updated: February 10, 2026 **Version:** 1.0.0 **Status:** Production Ready